

Database Systems

A Complete Guide

From first principles to distributed architectures.

Covering SQL, normalization, transactions, indexing, and NoSQL.

April 26, 2026

Contents

1	What Is a Database System?	3
1.1	DBMS Architecture	3
2	Data Models	3
3	The Relational Model	3
3.1	Key Types	3
3.2	Constraints	4
4	SQL — Structured Query Language	4
4.1	DDL — Data Definition Language	4
4.2	DML — Data Manipulation Language	5
4.3	DQL — Data Query Language	5
4.4	JOINS	6
4.5	Aggregate Functions	6
4.6	Window Functions	7
4.7	CTEs and Subqueries	7
4.8	DCL and TCL	7
5	Normalization	8
5.1	First Normal Form (1NF)	8
5.2	Second Normal Form (2NF)	8
5.3	Third Normal Form (3NF)	8
5.4	Boyce-Codd Normal Form (BCNF)	9
5.5	Higher Normal Forms	9
6	Transactions and ACID	9
6.1	Isolation Levels	9
7	Concurrency Control	10
7.1	Pessimistic Locking	10
7.2	Optimistic Concurrency Control (OCC)	10
7.3	MVCC — Multi-Version Concurrency Control	10
8	Indexing and Query Performance	10
8.1	B-Tree Index	10
8.2	Other Index Types	11
8.3	Reading an Explain Plan	11
9	Query Processing Pipeline	11
10	Advanced SQL Features	12
10.1	Views	12
10.2	Materialized Views	12
10.3	Stored Procedures and Functions	12
10.4	Triggers	13

11 Normalization vs Denormalization	13
12 NoSQL Databases	13
12.1 Key-Value Stores	14
12.2 Document Stores	14
12.3 Wide-Column Stores	14
12.4 Graph Databases	14
13 Distributed Databases and the CAP Theorem	14
13.1 The CAP Theorem	14
13.2 Scaling Strategies	15
14 Database Design Patterns	15
15 Choosing the Right Database	15
16 Quick Reference Summary	16

1. What Is a Database System?

A **database** is an organized collection of structured data. A **Database Management System (DBMS)** is the software layer that manages it—handling storage, retrieval, security, concurrency, and data integrity.

Why not just use files?

Plain files lack concurrent-access control, crash recovery, structured querying, and integrity enforcement. A DBMS provides all of these, making it the foundation of virtually every production application.

1.1. DBMS Architecture

A modern DBMS is organized into three layers:

1. **Query Processor** — parses SQL text into an abstract syntax tree, resolves names against the catalog, optimizes the plan, and schedules execution.
2. **Transaction Manager** — enforces ACID properties and handles concurrency control (locking, MVCC).
3. **Storage Engine** — manages the buffer pool (in-memory page cache), writes to data files on disk, and maintains the write-ahead log (WAL) for crash recovery.

Supporting all three layers are the **system catalog** (metadata about tables, columns, indexes, and users), the **write-ahead log** (WAL), and the **buffer pool** (pages of data held in RAM).

2. Data Models

A **data model** defines how data is structured, stored, and queried.

The **relational model** (Edgar Codd, 1970) dominates production systems. All non-relational models are commonly called *NoSQL*.

3. The Relational Model

A relational database organizes data into **relations** (tables). Each table has *rows* (tuples) and *columns* (attributes). Tables relate to one another through keys.

3.1. Key Types

Primary Key (PK)

Uniquely identifies every row. Must be unique and NOT NULL.

Foreign Key (FK)

A column referencing a PK in another table. Enforces *referential integrity*.

Model	Structure	Query language	Examples
Relational	Tables (rows + cols)	SQL	PostgreSQL, MySQL, SQLite
Document	JSON/BSON documents	MQL, aggregations	MongoDB, CouchDB
Key-Value	Key $\rightarrow$ Value pairs	GET/SET	Redis, DynamoDB
Wide-column	Column families	CQL	Cassandra, HBase
Graph	Nodes + Edges	Cypher, Gremlin	Neo4j, Neptune
Time-series	Timestamped values	Flux, PromQL	InfluxDB, TimescaleDB

Table 1: Major database data models

Candidate Key

Any minimal set of columns that could serve as a PK.

Surrogate Key

An artificial PK (e.g. `id SERIAL`) with no business meaning.

Natural Key

A PK drawn from real-world data (e.g. `email`, `SSN`).

Composite Key

A PK formed from two or more columns.

3.2. Constraints

- `NOT NULL` — column may not contain a null value.
- `UNIQUE` — all values in the column must be distinct.
- `CHECK` — a predicate every inserted or updated value must satisfy.
- `DEFAULT` — value used when none is supplied.
- `REFERENCES` — declares a foreign key relationship.

4. SQL — Structured Query Language

SQL is the universal interface for relational databases. It is divided into four sub-languages.

4.1. DDL — Data Definition Language

DDL creates and modifies the database structure.

```

1 -- Create a table
2 CREATE TABLE customers (
3   customer_id SERIAL PRIMARY KEY,
```

```
4  name          VARCHAR(100)    NOT NULL ,
5  email         VARCHAR(255)    UNIQUE NOT NULL ,
6  created_at    TIMESTAMP      DEFAULT NOW()
7 );
8
9  -- Add a column
10 ALTER TABLE customers ADD COLUMN phone VARCHAR(20);
11
12 -- Create an index
13 CREATE INDEX idx_email ON customers(email);
14
15 -- Remove a table permanently
16 DROP TABLE customers;
17
18 -- Remove all rows but keep structure
19 TRUNCATE TABLE orders;
```

Listing 1: DDL examples

4.2. DML — Data Manipulation Language

DML inserts, updates, and deletes rows.

```
1  -- Insert a single row
2  INSERT INTO customers (name, email)
3  VALUES ('Alice Tan', 'alice@example.com');
4
5  -- Insert multiple rows
6  INSERT INTO products (name, price, stock_qty) VALUES
7  ('Widget A', 9.99, 500),
8  ('Gadget B', 24.99, 200);
9
10 -- Update matching rows
11 UPDATE customers
12 SET   phone = '+1-800-555-0199'
13 WHERE customer_id = 42;
14
15 -- Delete rows
16 DELETE FROM orders
17 WHERE order_date < '2020-01-01';
```

Listing 2: DML examples

4.3. DQL — Data Query Language

The SELECT statement is the heart of SQL.

```
1  -- Basic SELECT
2  SELECT name, email
3  FROM   customers
4  WHERE  created_at > '2024-01-01'
5  ORDER BY name ASC
6  LIMIT 10;
7
```

```
8  -- Pattern matching and null check
9  SELECT * FROM customers
10 WHERE email LIKE '%@gmail.com'
11     AND phone IS NOT NULL;
12
13 -- CASE expression (computed column)
14 SELECT name,
15     CASE
16         WHEN total > 1000 THEN 'VIP'
17         WHEN total > 100  THEN 'Regular'
18         ELSE 'New'
19     END AS tier
20 FROM customers JOIN orders USING (customer_id);
```

Listing 3: SELECT with filtering and ordering

4.4. JOINS

```
1  -- INNER JOIN: only rows matched in both tables
2  SELECT c.name, o.order_date, o.total
3  FROM   customers c
4  INNER JOIN orders o ON c.customer_id = o.customer_id;
5
6  -- LEFT JOIN: all customers, even those with no orders
7  SELECT c.name, COUNT(o.order_id) AS order_count
8  FROM   customers c
9  LEFT JOIN orders o ON c.customer_id = o.customer_id
10 GROUP BY c.customer_id;
11
12 -- FULL OUTER JOIN: all rows from both sides
13 SELECT * FROM a FULL OUTER JOIN b ON a.id = b.id;
14
15 -- CROSS JOIN: every combination (Cartesian product)
16 SELECT * FROM sizes CROSS JOIN colors;
```

Listing 4: JOIN types

4.5. Aggregate Functions

```
1  -- Revenue summary grouped by month
2  SELECT
3      DATE_TRUNC('month', order_date) AS month,
4      COUNT(*)      AS order_count,
5      SUM(total)     AS revenue,
6      AVG(total)     AS avg_order,
7      MAX(total)     AS biggest
8  FROM   orders
9  GROUP BY month
10 HAVING SUM(total) > 5000 -- filter AFTER grouping
11 ORDER BY month;
```

Listing 5: Aggregates and GROUP BY

WHERE vs HAVING: WHERE filters individual rows *before* grouping. HAVING filters groups *after* aggregation. You cannot use aggregate functions in a WHERE clause.

4.6. Window Functions

Window functions aggregate *without* collapsing rows — each row retains its identity while gaining access to a computed value across a partition.

```

1 SELECT
2     name,
3     total,
4     RANK()      OVER (ORDER BY total DESC) AS rank,
5     SUM(total)  OVER ()                    AS grand_total,
6     LAG(total)  OVER (ORDER BY order_date) AS prev_total
7 FROM orders;
```

Listing 6: Window function examples

4.7. CTEs and Subqueries

```

1 -- Correlated subquery
2 SELECT name FROM customers
3 WHERE customer_id IN (
4     SELECT customer_id FROM orders WHERE total > 500
5 );
6
7 -- CTE (cleaner, reusable, readable)
8 WITH big_orders AS (
9     SELECT customer_id, SUM(total) AS lifetime_value
10    FROM orders
11   GROUP BY customer_id
12  HAVING SUM(total) > 1000
13 )
14 SELECT c.name, b.lifetime_value
15 FROM customers c
16 JOIN big_orders b USING (customer_id);
17
18 -- Recursive CTE for tree traversal (e.g. org chart)
19 WITH RECURSIVE org AS (
20     SELECT id, name, manager_id
21    FROM employees WHERE id = 1
22   UNION ALL
23     SELECT e.id, e.name, e.manager_id
24    FROM employees e
25   JOIN org o ON e.manager_id = o.id
26 )
27 SELECT * FROM org;
```

Listing 7: Subqueries and Common Table Expressions

4.8. DCL and TCL

```

1 -- Grant / revoke
```



```

2 GRANT SELECT, INSERT ON customers TO app_user;
3 REVOKE DELETE          ON orders FROM app_user;
4
5 -- Explicit transaction
6 BEGIN;
7 UPDATE accounts SET balance = balance - 100 WHERE id = 1;
8 UPDATE accounts SET balance = balance + 100 WHERE id = 2;
9 COMMIT; -- or ROLLBACK on failure
10
11 -- Savepoints
12 SAVEPOINT after_debit;
13 ROLLBACK TO after_debit;

```

Listing 8: Permissions and transaction control

5. Normalization

Normalization is the process of structuring a relational schema to **reduce redundancy** and prevent *update, insert, and delete anomalies*. Each Normal Form (NF) is a progressively stricter rule.

5.1. First Normal Form (1NF)

1NF Rule

Every cell holds exactly one atomic value. No repeating groups. Each row is uniquely identifiable by a primary key.

Violation example: A products column containing "Widget, Gadget" in a single cell is not atomic. Split into individual rows.

5.2. Second Normal Form (2NF)

2NF Rule

Must satisfy 1NF. Every non-key column must depend on the *entire* primary key — no partial dependencies.

Applies when the PK is composite. If CustPhone depends only on CustomerID but the PK is (OrderID, LineNo), extract customers to their own table.

5.3. Third Normal Form (3NF)

3NF Rule

Must satisfy 2NF. Every non-key column must depend *directly* on the primary key — no transitive dependencies ($A \rightarrow B \rightarrow C$ where B is a non-key attribute).

Example: If Price depends on Product and Product depends on the order key, move Price into a products table.

5.4. Boyce-Codd Normal Form (BCNF)

BCNF Rule

A stricter 3NF. For every functional dependency $X \rightarrow Y$, X must be a superkey. Handles anomalies involving overlapping candidate keys.

5.5. Higher Normal Forms

4NF No multi-valued dependencies (e.g. an employee having many skills *and* many languages stored in the same table).

5NF No join dependencies. Rarely encountered in practice.

Most production schemas target **3NF** and selectively *denormalize* for read performance (e.g. storing a pre-computed total on orders to avoid re-summing order lines on every read).

6. Transactions and ACID

A **transaction** is a logical unit of work that must succeed or fail as a whole — never partially. The four ACID properties guarantee correctness.

Property	Guarantee	Mechanism
Atomicity	All or nothing	ROLLBACK undoes partial work on failure.
Consistency	Valid state $\rightarrow$ valid state	Constraints and rules are never violated during a commit.
Isolation	Concurrent txns don't interfere	Enforced via locking or MVCC; controlled by isolation level.
Durability	Committed data survives crashes	Write-ahead log (WAL) + periodic checkpoints.

Table 2: The ACID properties

6.1. Isolation Levels

Higher isolation provides more correctness but more contention (lower throughput). Most databases default to **Read Committed**.

Dirty read Reading a value written by an uncommitted transaction.

Non-repeatable read The same row returns different values within the same transaction because another transaction committed a change.

Phantom read A re-executed range query returns a different set of rows because another

Isolation Level	Dirty Read	Non-repeatable Read	Phantom Read
Read Uncommitted	possible	possible	possible
Read Committed	prevented	possible	possible
Repeatable Read	prevented	prevented	possible
Serializable	prevented	prevented	prevented

Table 3: SQL isolation levels and the anomalies they prevent

transaction inserted or deleted rows.

7. Concurrency Control

7.1. Pessimistic Locking

Acquire a lock *before* reading or writing data.

- **Shared lock** (read lock): many transactions may hold it simultaneously; blocks exclusive locks.
- **Exclusive lock** (write lock): only one transaction may hold it; blocks all other locks.

Simple to reason about but creates contention under high concurrency.

7.2. Optimistic Concurrency Control (OCC)

Proceed without locking. At commit time, check whether any conflicting write occurred during execution and abort if so. Efficient when conflicts are rare (most web-application workloads).

7.3. MVCC — Multi-Version Concurrency Control

Used by PostgreSQL, MySQL InnoDB, and Oracle. Every write creates a new *version* of the row. Readers see a consistent snapshot from the start of their transaction. Readers never block writers; writers never block readers.

8. Indexing and Query Performance

An **index** is a separate data structure that accelerates lookups. Without an index, every query performs a *sequential scan* ($O(n)$). With a B-Tree index, point lookups become $O(\log n)$.

8.1. B-Tree Index

The default index type in every major RDBMS. Supports:

- Equality predicates: `WHERE id = 42`

- Range predicates: `WHERE price BETWEEN 10 AND 50`
- Sorting: `ORDER BY created_at DESC`
- Prefix matching: `WHERE name LIKE 'A1%'`

Leaf nodes are linked in a doubly-linked list, enabling efficient range scans. Internal nodes act as a sorted directory for binary search.

8.2. Other Index Types

Hash index $O(1)$ equality lookups. No range support. Useful for exact-match caches.

GIN (Generalized Inverted Index) Full-text search, arrays, JSONB containment queries.

GiST Geometric and geographic types (PostGIS, nearest-neighbor).

Partial index Indexes only rows matching a predicate, e.g. `WHERE active = true`. Smaller and faster.

Covering index Includes all columns a query needs (`INCLUDE` clause in PostgreSQL) — avoids a heap fetch.

Composite index Index on multiple columns. Column order matters: the index is only useful if the leftmost columns appear in the `WHERE` clause.

8.3. Reading an Explain Plan

```
1 EXPLAIN ANALYZE
2 SELECT * FROM orders
3 WHERE customer_id = 42 AND total > 100;
```

Listing 9: Analyzing a query plan

Key nodes to recognize:

- **Seq Scan** — full table scan. Often indicates a missing index.
- **Index Scan** — uses the index to find matching rows, then fetches from the heap.
- **Index Only Scan** — all needed columns are in the index; no heap access.
- **Bitmap Index Scan** — collects matching TIDs from the index then fetches heap pages in physical order. Efficient for low-selectivity conditions.
- **Hash Join / Merge Join / Nested Loop** — join strategies chosen by the optimizer based on row-count estimates.

9. Query Processing Pipeline

When you execute a SQL statement, the DBMS passes it through six stages:

1. **Parse** — tokenize and build an abstract syntax tree (AST). Raises syntax errors.

2. **Bind** (Semantic analysis) — resolve table and column names against the system catalog. Raises semantic errors.
3. **Optimize** — the query optimizer enumerates execution plans and selects the one with the lowest estimated cost using table statistics (row counts, column cardinalities, histograms).
4. **Plan** — output a physical operator tree (e.g. SeqScan → HashJoin → Sort → Limit).
5. **Execute** — operators are evaluated in a pipelined or volcano model, fetching pages from the buffer pool as needed.
6. **Return results** — rows are streamed back to the client.

The **cost-based optimizer** is the most sophisticated component. It uses statistics to estimate selectivity, choose join algorithms, and decide whether an index scan is cheaper than a sequential scan.

10. Advanced SQL Features

10.1. Views

A view is a saved query that behaves like a virtual table. It does not store data itself; the underlying query is re-executed on every access.

```
1 CREATE VIEW vw_active_customers AS
2 SELECT *
3 FROM   customers
4 WHERE  last_order > NOW() - INTERVAL '1 year';
```

Listing 10: Creating a view

10.2. Materialized Views

A materialized view physically stores the query result. It must be *refreshed* when the underlying data changes. Ideal for expensive aggregations that are queried frequently.

```
1 CREATE MATERIALIZED VIEW monthly_revenue AS
2 SELECT DATE_TRUNC('month', order_date) AS month,
3        SUM(total) AS revenue
4 FROM   orders
5 GROUP BY 1;
6
7 -- Refresh manually (or via a scheduled job)
8 REFRESH MATERIALIZED VIEW monthly_revenue;
```

Listing 11: Materialized view

10.3. Stored Procedures and Functions

```
1 CREATE FUNCTION get_customer_lifetime_value(cust_id INT)
2 RETURNS DECIMAL AS $$
```

```

3  SELECT SUM(total)
4  FROM    orders
5  WHERE   customer_id = cust_id;
6  $$ LANGUAGE SQL;
7
8  -- Call it
9  SELECT get_customer_lifetime_value(42);

```

Listing 12: SQL function

10.4. Triggers

Triggers execute automatically in response to INSERT, UPDATE, or DELETE events.

```

1  CREATE FUNCTION set_updated_at()
2  RETURNS TRIGGER AS $$
3  BEGIN
4      NEW.updated_at = NOW();
5      RETURN NEW;
6  END;
7  $$ LANGUAGE plpgsql;
8
9  CREATE TRIGGER update_timestamp
10 BEFORE UPDATE ON products
11 FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

Listing 13: Trigger to auto-update a timestamp

11. Normalization vs Denormalization

	Normalization	Denormalization
Goal	Eliminate redundancy	Optimize read performance
Write cost	Lower	Higher (more rows to update)
Read cost	Higher (joins needed)	Lower (pre-joined)
Consistency	Easier to maintain	Risk of stale duplicates
Use case	OLTP (transactional)	OLAP (analytical)

Table 4: Normalization vs denormalization trade-offs

12. NoSQL Databases

NoSQL databases trade some relational guarantees for scale, flexibility, or throughput. They follow the **BASE** model instead of ACID:

- **Basically Available** — the system responds to every request, though the response may be stale.
- **Soft state** — the system state may change over time even without new input (due to eventual consistency).
- **Eventually consistent** — replicas will converge to the same state given enough time and no new writes.

12.1. Key-Value Stores

Examples: Redis, DynamoDB, Riak.

Structure: a globally distributed hash table of `key` → `value` pairs. Values can be strings, lists, hashes, sets, or sorted sets (Redis).

Best for: sessions, caching, rate limiting, leaderboards, pub/sub queues, feature flags.

12.2. Document Stores

Examples: MongoDB, CouchDB, Firestore.

Structure: semi-structured JSON or BSON documents grouped into *collections*. Documents in the same collection need not share a schema.

Best for: product catalogs, CMS, user profiles, applications with rapidly evolving schemas.

12.3. Wide-Column Stores

Examples: Apache Cassandra, HBase, ScyllaDB.

Structure: rows identified by a partition key; each row can have a different set of columns. Optimized for write-heavy workloads distributed across many nodes.

Best for: time-series data, IoT telemetry, event logs, anything requiring linear horizontal write scalability.

12.4. Graph Databases

Examples: Neo4j, Amazon Neptune.

Structure: nodes (entities) and edges (relationships), both with typed properties. Relationship traversal is $O(1)$ per hop rather than $O(\log n)$ for a join.

Best for: social networks, fraud detection, recommendation engines, knowledge graphs, any workload where relationship depth is the query.

13. Distributed Databases and the CAP Theorem

13.1. The CAP Theorem

Brewer (2000) proved that in a distributed system subject to network partitions, you can guarantee at most two of the following three properties:

Consistency (C) Every read receives the most recent committed write.

Availability (A) Every request receives a response (not necessarily the latest data).

Partition tolerance (P) The system continues operating despite network partitions between nodes.

Because network partitions are unavoidable in practice, the real-world choice is:

- **CP** systems — sacrifice availability during a partition to maintain consistency. Examples: HBase, Zookeeper, etcd.
- **AP** systems — remain available during a partition but may serve stale data. Examples: Cassandra, CouchDB, DynamoDB (with eventual consistency).

13.2. Scaling Strategies

Vertical scaling Larger machine, more RAM, faster disks. Simple but has a hard ceiling.

Replication Copies of data on multiple nodes. Reads can be distributed across replicas. Leader-follower (primary-replica) is the most common topology.

Sharding Each shard holds a subset of rows, partitioned by key range or hash. Enables linear write scaling.

CQRS Command Query Responsibility Segregation — separate write model (normalized, ACID) from read model (denormalized, fast).

Connection pooling
Reuse expensive database connections via a pool (PgBouncer, HikariCP). Critical at scale.

14. Database Design Patterns

15. Choosing the Right Database

1. **Structured, relational data with complex queries?** → PostgreSQL or MySQL.
2. **Flexible/nested document structure with evolving schema?** → MongoDB.
3. **Massive write throughput at global scale?** → Cassandra or DynamoDB.
4. **Caching, sessions, real-time leaderboards?** → Redis.
5. **Highly connected data / relationship-centric queries?** → Neo4j.
6. **Timestamped metrics and sensor data?** → InfluxDB or TimescaleDB.
7. **OLAP, analytics, data warehousing?** → Snowflake, BigQuery, or ClickHouse.

Polyglot persistence — many production systems use multiple databases: PostgreSQL for core transactional data, Redis for caching, Elasticsearch for full-text search, and a

Pattern	Description
Soft delete	Add <code>deleted_at</code> <code>TIMESTAMP</code> instead of physically removing rows. Preserves history and enables undo.
Audit log table	Shadow table tracking every mutation with <code>changed_by</code> and <code>changed_at</code> . Append-only.
Temporal tables	Store full row history with <code>valid_from/valid_to</code> columns. Supports <code>AS OF</code> queries.
Partitioning	Divide a large table into physical sub-tables by range (e.g. monthly) or list. Transparent to queries.
Read replicas	Route heavy <code>SELECT</code> workloads to replica nodes; route writes to the primary.
EAV	Entity-Attribute-Value — ultra-flexible but hard to query and type-check. Last resort for truly sparse schemas.
Polymorphic FK	One FK column references multiple tables. Avoid when possible; breaks referential integrity.

Table 5: Common database design patterns

data warehouse for analytics.

16. Quick Reference Summary

Topic	Key takeaway
Relational model	Data in tables; relationships via foreign keys; integrity via constraints.
SQL sub-languages	DDL (structure), DML (rows), DQL (queries), DCL (access), TCL (transactions).
Normalization	1NF $\rightarrow$ atomic; 2NF $\rightarrow$ no partial deps; 3NF $\rightarrow$ no transitive deps. Target 3NF.
ACID	Atomicity, Consistency, Isolation, Durability — non-negotiable for financial and correctional workloads.
Isolation levels	Weaker = faster, stronger = safer. Default: Read Committed.
MVCC	Readers never block writers. The standard in PostgreSQL, MySQL InnoDB, Oracle.
B-Tree index	Default; covers equality, range, sort. Use <code>EXPLAIN ANALYZE</code> to verify usage.
NoSQL	BASE instead of ACID. Pick model to match access pattern, not familiarity.
CAP theorem	CP or AP — pick based on whether your domain tolerates stale reads or unavailability more.
Scaling	Replicate for read scale; shard for write scale; pool connections always.

Table 6: Database systems at a glance